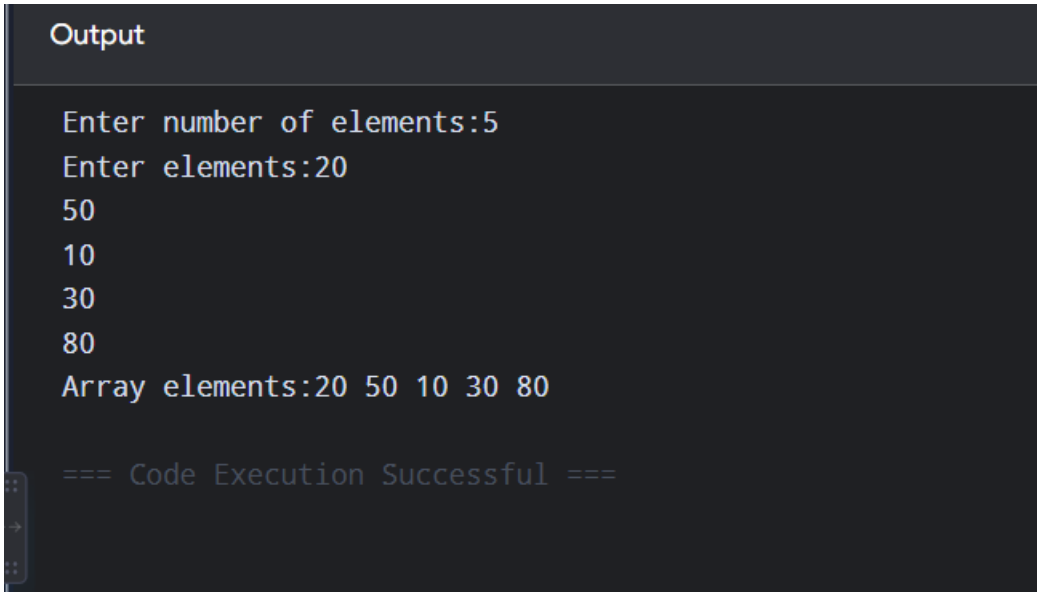
DATA STRUCTURE ASSIGNMENT 2

1)Define an integer array. Declare an integer pointer and initialize it to point to the first element of the array. Use the pointer to traverse the array and print the elements

CODE:

```
#include<stdio.h>
int main(){
    int arr[100];
    int n,i;
    printf("Enter number of elements:");
    scanf("%d",&n);
    int*ptr=arr;
    printf("Enter elements:");
    for(i=0;i<n;i++)
        scanf("%d",&arr[i]);
    printf("Array elements:");
    for(i=0;i<n;i++)
        printf("%d ",*(ptr+i));
    return 0;
}
```

OUTPUT:

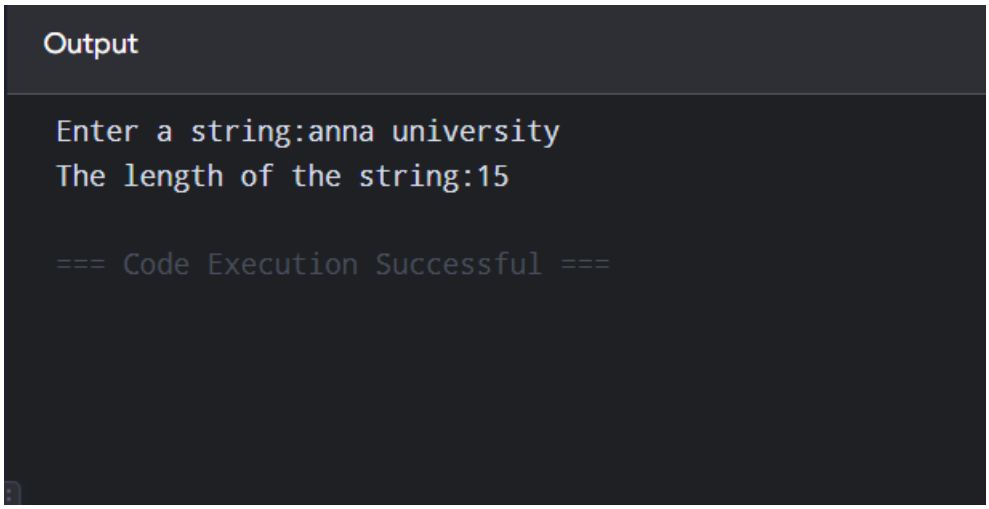


2)Calculate string length using pointers

CODE:

```
#include<stdio.h>
int main(){
    char str[100];
    char*p;
    int length=0;
    printf("Enter a string:");
    fgets(str,100,stdin);
    p=str;
    while(*p!='\0'){
        if(*p!='\n')
            length++;
        p++;
    }
    printf("The length of the string:%d",length);
    return 0;
}
```

OUTPUT:



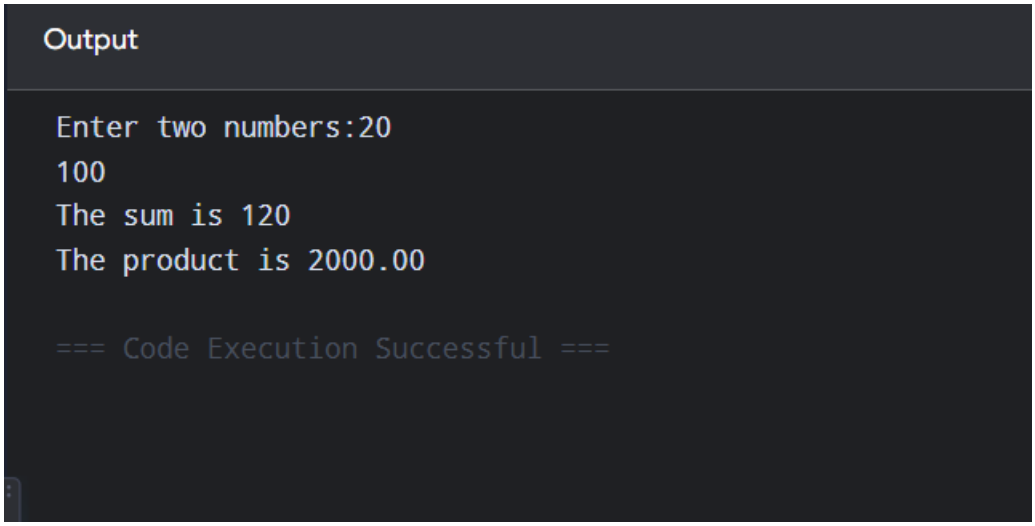
3)Write a function that calculates both the sum and product of two numbers. Return these two results using pointers passed as arguments to the functions.

CODE:

```
#include<stdio.h>
void calculate(int*a,int*b,int*sum,float*pro){
    *sum=*a+*b;
    *pro=(*a)*(*b);
}

int main(){
    int a,b,sum;
    float pro;
    printf("Enter two numbers:");
    scanf("%d %d",&a,&b);
    calculate(&a,&b,&sum,&pro);
    printf("The sum is %d\n",sum);
    printf("The product is %.2f",pro);
    return 0;
}
```

OUTPUT:



4)Create an array of structures. Use dynamic memory allocation to allocate memory space and use a structure pointer with pointer arithmetic to iterate and access/modify memebers

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Student
{
    int id;
    char name[50];
    float marks;
};

int main()
{
    int n;

    printf("Enter number of students: ");
    scanf("%d", &n);
```

```
struct Student *p;

p = (struct Student *)malloc(n * sizeof(struct Student));

if (p == NULL)
{
    printf("Memory allocation failed");
    return 1;
}

// Input details
for (int i = 0; i < n; i++)
{
    printf("\nEnter details of student %d:\n", i + 1);

    printf("ID: ");
    scanf("%d", &(p + i)->id);

    printf("Name: ");
    scanf("%s", (p + i)->name);

    printf("Marks: ");
    scanf("%f", &(p + i)->marks);
}

// Display details
printf("\nStudent Details:\n");
```

```
    for (int i = 0; i < n; i++)
    {
        printf("ID: %d\n", (p + i)->id);
        printf("Name: %s\n", (p + i)->name);
        printf("Marks: %.2f\n", (p + i)->marks);
    }

    // Modify first student's marks
    (p + 0)->marks = 95;

    printf("\nAfter modification:\n");
    printf("ID: %d\n", (p + 0)->id);
    printf("Name: %s\n", (p + 0)->name);
    printf("Marks: %.2f\n", (p + 0)->marks);

    free(p);

    return 0;
}
```

OUTPUT:

Output

```
Enter number of students: 2

Enter details of student 1:
ID: 18
Name: ram
Marks: 90

Enter details of student 2:
ID: 16
Name: rekha
Marks: 100

Student Details:
ID: 18
Name: ram
Marks: 90.00
ID: 16
Name: rekha
Marks: 100.00

After modification:
ID: 18
Name: ram
Marks: 95.00
```

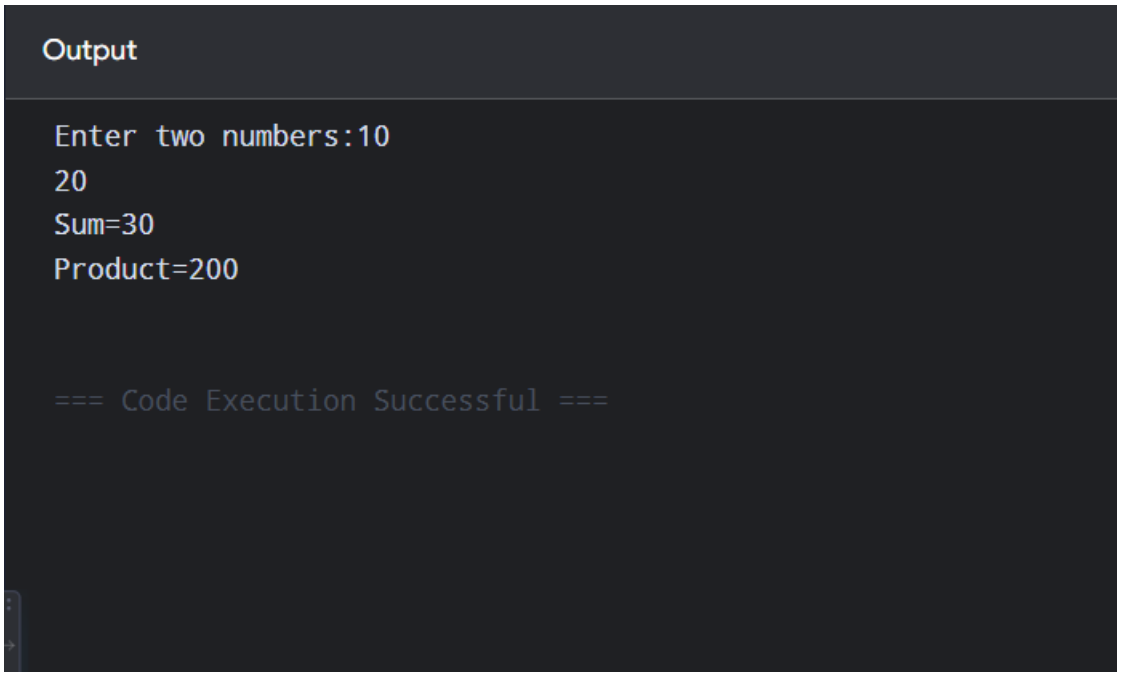
5)define two functions with same signature. Declare a function pointer and use the pointer with pointer arithmetic.

CODE:

```
#include<stdio.h>
void add(int a,int b){
    printf("Sum=%d\n",a+b);
}
void multiply(int a,int b){
    printf("Product=%d\n",a*b);
}
int main(){
    int a,b;
```

```
    printf("Enter two numbers:");
    scanf("%d %d",&a,&b);
    void(*fp)(int,int);
    fp=add;
    fp(a,b);
    fp=multiply;
    fp(a,b);
    return 0;
}
```

OUTPUT:



6)Declare an integer, a pointer to it and a double pointer. Print the values and addresses at each level .

CODE:

```
#include <stdio.h>

int main() {
    int x = 10;

    int *p = &x;
```

```
    int **q = &p;

    printf("Value of x = %d\n", x);
    printf("Address of x = %p\n", (void *)&x);

    printf("\nUsing pointer p:\n");
    printf("Value = %d\n", *p);
    printf("Address stored in p = %p\n", (void *)p);
    printf("Address of p = %p\n", (void *)&p);

    printf("\nUsing double pointer q:\n");
    printf("Value = %d\n", **q);
    printf("Address stored in q = %p\n", (void *)q);
    printf("Address of q = %p\n", (void *)&q);

    return 0;
}
```

OUTPUT:



7)Sort an array of floating point numbers using selection sort.

CODE:

```
#include <stdio.h>

int main() {
    float arr[5];
    int i, j, min;
    float temp;

    printf("Enter 5 floating-point numbers:\n");
    for (i = 0; i < 5; i++)
        scanf("%f", &arr[i]);

    for (i = 0; i < 4; i++) {
        min = i;
```

```
        for (j = i + 1; j < 5; j++) {
            if (arr[j] < arr[min])
                min = j;
        }

        temp = arr[i];
        arr[i] = arr[min];
        arr[min] = temp;
    }

    printf("Sorted array:\n");
    for (i = 0; i < 5; i++)
        printf("%.2f ", arr[i]);

    return 0;
}
```

OUTPUT:

```
Output

Enter 5 floating-point numbers:
11.00
48.23
23.23
23.25
54.023
Sorted array:
11.00 23.23 23.25 48.23 54.02

=== Code Execution Successful ===
```

8)Define a structure student with id, name and grade. Create an array of structure and sort the them based on different fields.

CODE:

```
#include <stdio.h>
#include <string.h>

struct Student {
    int id;
    char name[50];
    float grade;
};

void sortById(struct Student s[], int n) {
    struct Student temp;

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (s[j].id > s[j + 1].id) {
                temp = s[j];
                s[j] = s[j + 1];
                s[j + 1] = temp;
            }
        }
    }
}

void sortByGrade(struct Student s[], int n) {
    struct Student temp;
```

```
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (s[j].grade > s[j + 1].grade) {
                temp = s[j];
                s[j] = s[j + 1];
                s[j + 1] = temp;
            }
        }
    }
}

void display(struct Student s[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d  %s  %.2f\n",
               s[i].id, s[i].name, s[i].grade);
}

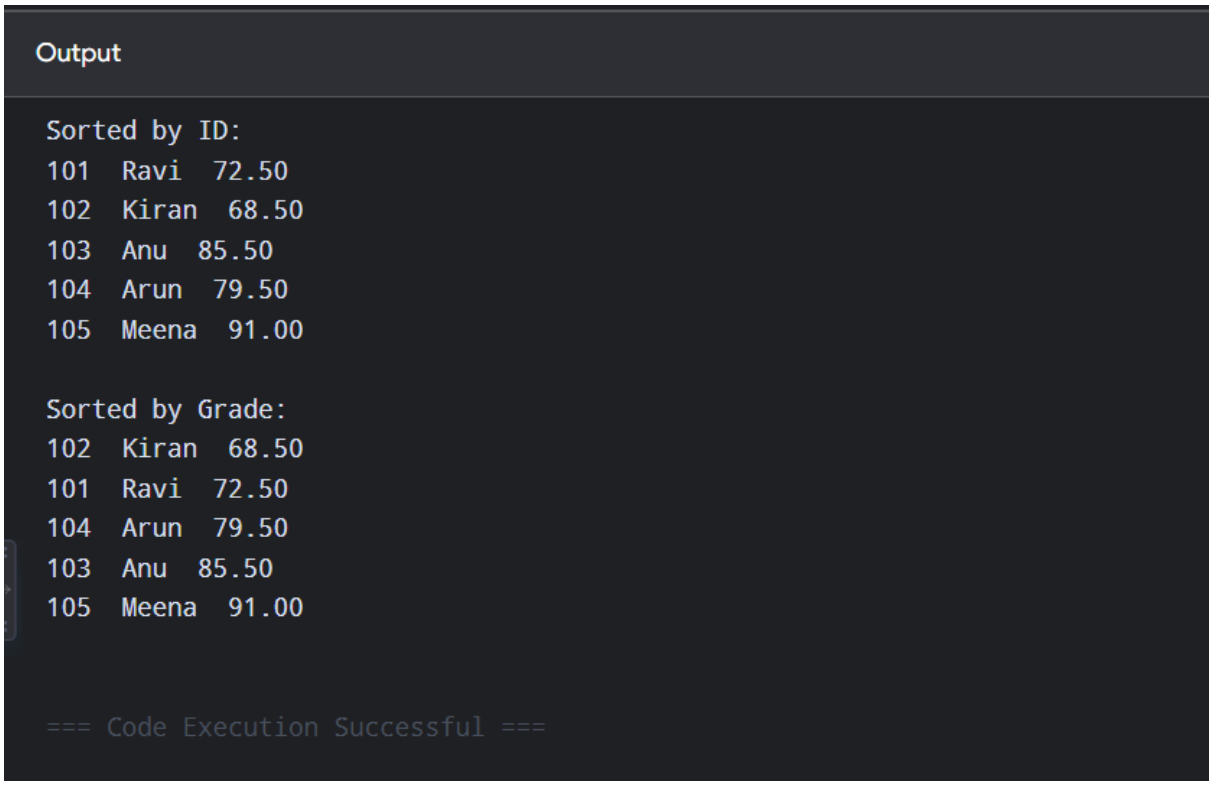
int main() {
    struct Student s[5] = {
        {103, "Anu", 85.5},
        {101, "Ravi", 72.5},
        {105, "Meena", 91.0},
        {102, "Kiran", 68.5},
        {104, "Arun", 79.5}
    };

    printf("Sorted by ID:\n");
    sortById(s, 5);
    display(s, 5);
```

```
    printf("\nSorted by Grade:\n");
    sortByGrade(s, 5);
    display(s, 5);

    return 0;
}
```

OUTPUT:



9)Sort a set of negative numbers using radix sort.

CODE:

```
#include <stdio.h>

int getMin(int a[], int n)
{
    int min = a[0];
```

```
    for (int i = 1; i < n; i++)
    {
        if (a[i] < min)
            min = a[i];
    }

    return min;
}

int getMax(int a[], int n)
{
    int max = a[0];

    for (int i = 1; i < n; i++)
    {
        if (a[i] > max)
            max = a[i];
    }

    return max;
}

void radixSort(int a[], int n)
{
    int offset = getMin(a, n);

    // Subtract offset
    for (int i = 0; i < n; i++)
```

```
    a[i] = a[i] - offset;

int max = getMax(a, n);
int output[n];

// Radix Sort
for (int place = 1; max / place > 0; place = place * 10)
{
    int count[10] = {0};

    for (int i = 0; i < n; i++)
        count[(a[i] / place) % 10]++;

    for (int i = 1; i < 10; i++)
        count[i] = count[i] + count[i - 1];

    for (int i = n - 1; i >= 0; i--)
    {
        int digit = (a[i] / place) % 10;

        output[count[digit] - 1] = a[i];
        count[digit]--;
    }

    for (int i = 0; i < n; i++)
        a[i] = output[i];
}

// Add offset back
```

```
    for (int i = 0; i < n; i++)
        a[i] = a[i] + offset;
}

int main()
{
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int a[n];

    printf("Enter %d numbers:\n", n);

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

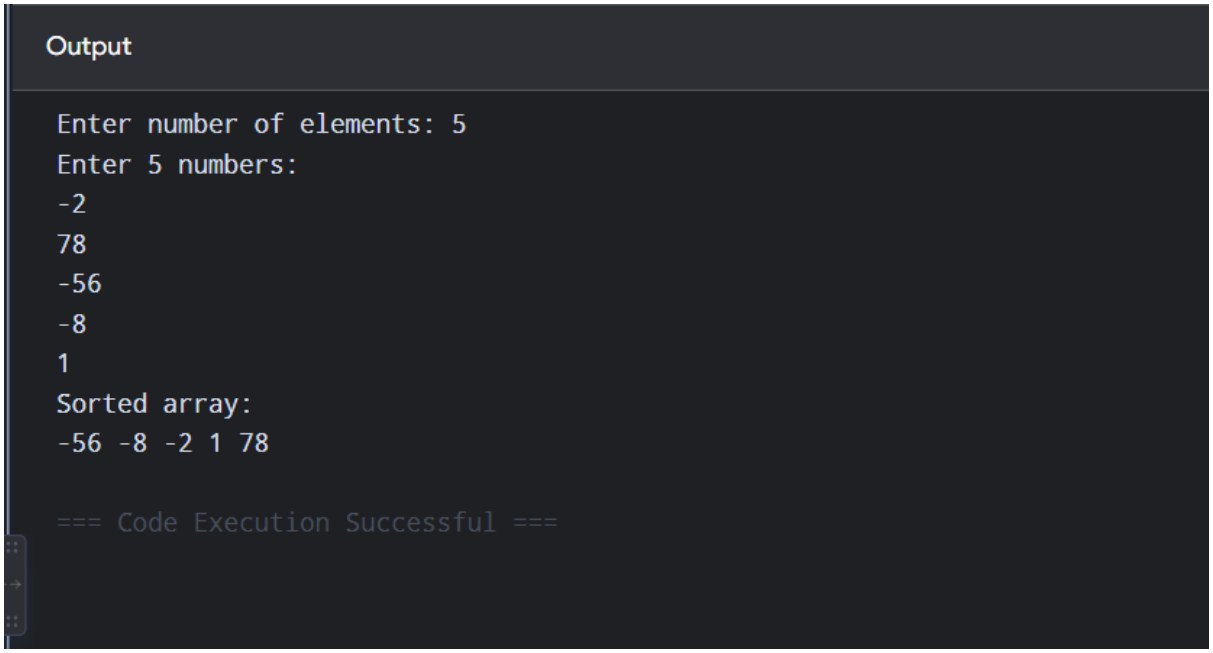
    radixSort(a, n);

    printf("Sorted array:\n");

    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

OUTPUT:



10)Write a C program to copy one array to another using pointer.

CODE:

```
#include <stdio.h>

int main() {
    int source[5], destination[5];
    int *p, *q;

    printf("Enter 5 elements:\n");
    for (int i = 0; i < 5; i++)
        scanf("%d", &source[i]);

    p = source;
    q = destination;

    for (int i = 0; i < 5; i++)
        *(q + i) = *(p + i);

    printf("Copied array:\n");
```

```
    for (int i = 0; i < 5; i++)
        printf("%d ", *(q + i));

    return 0;
}
```

OUTPUT:

